

1. Library Statistic Dashboard (Lua)

- 1.1. Objective: Design, build, and deploy an interactive Library Statistic Dashboard that consolidates key library performance metrics into a single, visual, user-friendly interface. The dashboard will support data-driven decision-making, reporting to stakeholders, collection assessment, advocacy, and operational insights.
- 1.2. Typical Scope (to be refined during discovery)
 - 1.2.1. Core KPIs: Circulation (loans, renewals, returns), physical & electronic resource usage, collection size/growth/turnover, gate counts / visitors, unique active patrons, program attendance, interlibrary loan / requests, digital resource usage.
 - 1.2.2. Natural Language Query (Optional)
 - 1.2.3. Visualizations: Trend charts, KPI cards, heatmaps, top lists, filters by date, material type, location, patron group, etc.
 - 1.2.4. Data sources: Data API provided by SYSIT team.
 - 1.2.5. Delivery options: ASP.Net Web App
 - 1.2.6. Users: Library staff
 - 1.2.7. Phase 1 – Discovery
 - Study the data source and current statistic platform.
 - Decide tech stack with Lib IT staff.
 - 1.2.8. Phase 2 – Design
 - Create interactive mockups (Figma or similar).
 - Define filters, drill-downs
 - 1.2.9. Phase 3 – Development
 - Implement main overview dashboard + specialized views (Circulation, Collections, Digital Resources, Users & Engagement).
 - Add Natural Language Query (NLQ) to display the statistic result if possible

1.3. Project Phases & High-Level Timeline

Phase	Duration	Key Activities	Deliverables
1. Discovery & Requirements	1 weeks	Data source inventory, KPI prioritization, tool selection	Prioritized KPI list, architecture recommendation
2. Design	1 weeks	Wireframes / mockups, data model design, visualization standards	Approved UI/UX designs, data model, technical design document
3. Dashboard Development	2 weeks	Build interactive dashboard(s), KPI cards, filters, charts, export	Functional MVP dashboard

Phase	Duration	Key Activities	Deliverables
		features, role-based views	
4. Testing, Pilot & Refinement	1 weeks	UAT with library staff, performance & data accuracy checks, bug fixes	Test report, refined dashboard, user feedback summary
5. Deployment, Training & Handover	1 weeks	Production deployment, documentation, training sessions, knowledge transfer	Live dashboard, user guides, training materials

2. Library E-Book OCR Project (Lua + Zachary)

- 2.1. Objective: Build a web application that allows users to upload scanned (image-based) PDFs. The system uses AI/OCR to extract text and structure, then generates a clean, searchable EPUB file.
- 2.2. Typical Scope PDF upload (drag-and-drop)
- 2.2.1. Background conversion job with status tracking
 - 2.2.2. Downloadable searchable EPUB
 - 2.2.3. User job history dashboard
 - 2.2.4. Basic error handling and progress indication
 - 2.2.5. Out of Scope
 - Advanced multi-column/magazine layouts
 - Handwriting recognition
 - Bulk API
 - User-editable intermediate results
 - Cloud object storage (will use local folders)

2.3. Final Tech Stack

Component	Technology	Purpose
Frontend + API	ASP.NET Core 8/9 Razor Pages	UI, Auth, Upload, Status, Download
Authentication	Not required	Not required
Database	SQLite	Jobs, metadata
Background Worker	Python 3.11+ (Windows Service)	OCR + AI + EPUB generation
OCR / Text Cleanup	Azure OpenAI / DeepSeek	High-quality text + structure extraction
EPUB Generation	Python ebooklib	Create searchable EPUB
Hosting	IIS (ASP.NET) + Python Windows	Single Windows server

Component	Technology	Purpose
	Service	recommended

2.3.1. Week 1: Foundation

- Create ASP.NET Core Razor Pages project
 - Configure EF Core + SQLite
 - Build PDF upload page and save file to Uploads folder
 - Create Job record with Status = "Pending"
 - Basic site layout and navigation
- Milestone: User can upload a PDF, and see the job created in SQLite.

2.3.2. Week 2: Python Worker + OCR

- Set up Python project and virtual environment
 - Connect Python to the same SQLite database
 - Build worker that polls for Pending jobs
 - Integrate Azure OpenAI / DeepSeek for OCR + layout analysis
 - Extract text and update job status to "Processing"
- Milestone: Uploaded scanned PDF is successfully processed by Python and text is extracted.

2.3.3. Week 3-4: Structure Cleanup + EPUB Generation

- Add LLM post-processing (cleanup + heading/paragraph detection)
 - Generate searchable EPUB using ebooklib (with TOC, metadata, basic CSS)
 - Save EPUB to Results folder
 - Update Job status = "Completed" and store result path
 - Create download functionality in ASP.NET
- Milestone: Full end-to-end conversion works (PDF → searchable EPUB).

2.3.4. Week 5: Dashboard & User Experience

- User dashboard showing all jobs with status and download button
 - Simple progress indicator (polling)
 - Error handling and failed job display
 - File validation, size limits, and basic security
- Milestone: Clean and usable end-to-end user flow.

2.3.5. Week 6: Testing, Quality & Hardening

- Test with 10-20 real scanned PDFs of varying quality
 - Improve OCR accuracy and EPUB structure
 - Handle large files and edge cases
- Milestone: Stable, reliable conversions with good output quality.

3. Library Chatbot link to PURE system (Zachary)

Description: Design and build a custom Python-based Model Context Protocol (MCP) server that exposes Elsevier Pure API data to LLMs. Integrate the MCP server into an n8n orchestration workflow to power an interactive, AI-driven chatbot

capable of querying academic and research records in real time.

3.1. Phase 1: Foundation & API Discovery (Weeks 1–4)

3.1.1. Week 1: Setup & Protocol Basics

- Set up the Python environment (using standard MCP Python SDK).
- Read the Model Context Protocol (MCP) specifications.
- *Deliverable:* Working "Hello World" local MCP server running via Studio.

3.1.2. Week 2: Elsevier Pure API Exploration

- Obtain Pure API keys and test endpoints using Postman/cURL (e.g., /research-outputs, /persons, /organisations).
- Inspect nested REST responses to determine what raw data needs filtering for an LLM context window.
- *Deliverable:* Postman collection verifying Pure API authentication and payload structures.

3.1.3. Week 3: Tool Architecture & Schema Design

- Define target MCP tools (e.g., search_publications, get_author_profile).
- Design input JSON schemas (Pydantic models in Python) so the LLM knows what arguments to provide.
- *Deliverable:* Architectural spec mapping Pure API endpoints to MCP tool definitions.

3.1.4. Week 4: Core Server Implementation

- Implement Python code to execute HTTP calls to Pure API when MCP tools are invoked.
- Format bulky Pure API responses into clean Markdown or optimized JSON for LLM parsing.
- *Deliverable:* Functional local MCP server handling 2–3 read-only operations via Stdio.

3.2. Phase 2: Server Hardening & Deployment (Weeks 5–7)

3.2.1. Week 5: Advanced Features & Local Validation

- Add query filtering parameters (date ranges, organizational units, pagination).
- Test and debug tool execution locally using an inspector client or Claude Desktop.
- *Deliverable:* Fully functional Python MCP server with local LLM testing completed.

3.2.2. Week 6: Web Transport Layer (Crucial for n8n)

- Expose the MCP server via **Server-Sent Events (SSE)** or **Streamable HTTP** endpoints (n8n requires network transport to connect to an external MCP server, not Stdio).
 - Implement bearer token or header-based authentication on the endpoint.
 - *Deliverable:* MCP server reachable locally via HTTP/SSE with auth.
- 3.2.3. Week 7: Containerization & Target 1 Delivery
- Create a Dockerfile for the Python MCP server.
 - Deploy the container to internal server or cloud VM environment.
 - *Deliverable: Target 1 Complete* — Deployed, production-ready Pure MCP server running over HTTP/SSE.
- 3.3. Phase 3: n8n & Chatbot Integration (Weeks 8–10)
- 3.3.1. Week 8: n8n Integration
- Set up n8n and configure the **MCP Client Tool** node.
 - Connect n8n to the deployed MCP Server URL and verify automatic tool discovery.
 - *Deliverable:* n8n workflow successfully listing and invoking Pure MCP tools via an AI Agent node.
- 3.3.2. Week 9: Chatbot Frontend Wiring
- Connect the chat interface (Slack, Teams, or Web UI) to the n8n webhook/chat trigger.
 - Implement system prompt instructions inside n8n to guide how the AI Agent formats research output.
 - *Deliverable:* Working prototype: User asks a question → Chatbot → n8n AI Agent → Pure MCP Server → Pure API → Answer sent back.
- 3.3.3. Week 10: Testing, Security & Handover
- Perform multi-turn conversation testing (handling missing inputs, API errors, and edge cases).
 - Conduct a security audit to ensure Pure API credentials and n8n tokens aren't leaked in logs.
 - Write technical setup documentation and deliver final project demo.
 - *Deliverable:* Fully documented project repository, merged code, and live demo.